数据结构：算法分析 (Algorithm Analysis) 复习讲义

目录

1 算法与分析对象 (Algorithms and What to Analyze) 3
1.1 算法的定义 . . . 3
1.2 选择排序示例 . . . 3
1.3 分析什么 . . . 3
1.4 计数示例 . . . 4

2 渐进记号 (Asymptotic Notation) 5
2.1 为什么使用渐进分析 . . . 5
2.2 渐进记号的四种形式 . . . 5
2.3 渐进记号规则 . . . 6
2.4 常见增长率顺序 . . . 6

3 一般分析规则 (General Rules) 6
3.1 递归示例：Fibonacci . . . 7

4 比较算法：最大子序列和 (Maximum Subsequence Sum) 7
4.1 问题定义 . . . 7
4.2 算法 1：三重循环 . . . 7
4.3 算法 2：二重循环并维护当前和 . . . 8
4.4 算法 3：分治 . . . 8
4.5 算法 4：在线算法 (On-line Algorithm) . . . 8
4.6 四种算法对比 . . . 9

5 运行时间中的对数 (Logarithms in the Running Time) 9
5.1 二分查找 . . . 9
5.2 欧几里得算法 (Euclid's Algorithm) . . . 10
5.3 快速幂 (Exponentiation by Squaring) . . . 11

6 检查复杂度分析 (Checking Your Analysis) **12**
6.1 倍增法 12
6.2 归一化检查 12

7 全章总结 **12**
7.1 复习主线 12

1 算法与分析对象 (Algorithms and What to Analyze)

1.1 算法的定义

算法 (Algorithm) 是一组有限的指令。如果按照这些指令执行，就能完成某个特定任务。一个算法必须满足五个条件：

1. **输入 (Input)**：有零个或多个外部给定的量。
2. **输出 (Output)**：至少产生一个量。
3. **确定性 (Definiteness)**：每条指令清楚且无歧义。
4. **有穷性 (Finiteness)**：对所有情况，算法都在有限步后终止。
5. **有效性 (Effectiveness)**：每条指令都足够基本，原则上能用纸笔手工完成；不仅要明确，还要可行。

算法与程序的区别

- 程序用某种编程语言写成，不一定满足有穷性。例如操作系统通常长期运行。
- 算法可以用自然语言、流程图、程序语言或伪代码描述。
- 课件中的 Selection Sort 用伪代码表达：从尚未排序部分找最小元素，再与当前位置交换。

1.2 选择排序示例

对 $n \geq 1$ 个整数按递增顺序排序：

```
for ( i = 0; i < n; i++ ) {
    examine list[i] to list[n-1];
    suppose the smallest integer is at list[min];
    interchange list[i] and list[min];
}
```

可理解为：**找最小值 + 与当前位置交换**。分析时还要考虑数据存在哪里、如何访问和交换。

1.3 分析什么

真实运行时间依赖机器、编译器、系统负载等因素。算法分析通常关心：

- **时间复杂度 (Time Complexity)**：输入规模增长时运行时间如何增长。
- **空间复杂度 (Space Complexity)**：算法额外使用的存储空间如何增长。

课件默认假设：

1. 指令顺序执行。
2. 每条简单指令耗费一个时间单位。
3. 整数大小固定，并且可用内存无限。

通常分析两个函数：

- $T_{\text{avg}}(N)$：平均情况时间复杂度。
- $T_{\text{worst}}(N)$：最坏情况时间复杂度。

如果输入不止一个维度，复杂度函数也可以有多个参数，例如矩阵加法的 $T(\text{rows}, \text{cols})$。

1.4 计数示例

矩阵加法

```
void add( int a[][MAX_SIZE], int b[][MAX_SIZE],
          int c[][MAX_SIZE], int rows, int cols )
{
    int i, j;
    for ( i = 0; i < rows; i++ )
        for ( j = 0; j < cols; j++ )
            c[i][j] = a[i][j] + b[i][j];
}
```

精确计数可得

$$T(\text{rows}, \text{cols}) = 2\text{rows} \cdot \text{cols} + 2\text{rows} + 1,$$

渐进地记为

$$T(\text{rows}, \text{cols}) = \Theta(\text{rows} \cdot \text{cols}).$$

求和：迭代与递归

迭代求和：

```
float sum( float list[], int n )
{
    float tempsum = 0;
    int i;
    for ( i = 0; i < n; i++ )
        tempsum += list[i];
    return tempsum;
}
```

课件给出的计数为 $T_{\text{sum}}(n) = 2n + 3$。

递归求和：

```
float rsum( float list[], int n )
{
    if ( n )
        return rsum( list, n-1 ) + list[n-1];
    return 0;
}
```

课件给出的计数为 $T_{\text{rsum}}(n) = 2n + 2$。虽然形式上少一个常数，但递归每步开销通常更大，因此不要只盯精确步数的常数项。

2 渐进记号 (Asymptotic Notation)

2.1 为什么使用渐进分析

逐条数步数的目标不是得到某台机器上的绝对秒数，而是预测输入规模 N 增大时运行时间的增长趋势。若

$$T_{p1}(N) = c_1 N^2 + c_2 N, \qquad T_{p2}(N) = c_3 N,$$

无论常数 c_1, c_2, c_3 如何，只要 N 足够大，二次函数最终会超过一次函数。

2.2 渐进记号的四种形式

四种渐进记号

- $T(N) = O(f(N))$：存在正常数 c, n_0，使得对所有 $N \geq n_0$，有

  $$T(N) \leq cf(N).$$

  表示渐进上界。

- $T(N) = \Omega(g(N))$：存在正常数 c, n_0，使得对所有 $N \geq n_0$，有

  $$T(N) \geq cg(N).$$

  表示渐进下界。

- $T(N) = \Theta(h(N))$ 当且仅当 $T(N) = O(h(N))$ 且 $T(N) = \Omega(h(N))$。

- $T(N) = o(p(N))$：$T(N) = O(p(N))$ 但 $T(N) \neq \Theta(p(N))$，表示严格低阶。

课件强调：

- 对 O 记号，总是选最小的有意义上界。例如 $2N + 3 = O(N)$。
- 对 Ω 记号，总是选最大的有意义下界。例如 $2N + N^2 = \Omega(N^2)$。

2.3 渐进记号规则

若 $T_1(N) = O(f(N))$ 且 $T_2(N) = O(g(N))$，则：

- $T_1(N) + T_2(N) = \max(O(f(N)), O(g(N)))$。
- $T_1(N) \cdot T_2(N) = O(f(N)g(N))$。

若 $T(N)$ 是 k 次多项式，则 $T(N) = \Theta(N^k)$。

对任意常数 k，有

$$\log^k N = O(N).$$

这说明对数函数增长非常慢。

2.4 常见增长率顺序

从慢到快常见顺序为：

$$1 < \log N < N < N \log N < N^2 < N^3 < 2^N < N!.$$

课件的图像比较了 $\log N$、N、$N \log N$、N^2 与 2^N，核心结论是：当 N 足够大时，指数和阶乘增长会迅速变得不可接受。

复杂度	1	$\log N$	N	$N \log N$	N^2	N^3	2^N
增长感受	常数	极慢	线性	近线性	平方	立方	指数
典型场景	直接访问	二分	扫描	排序	双循环	三循环	穷举子集

注意：比较渐进复杂度时必须保证 N 足够大。课件举例 $T_{p1}(N) = 10^6 N$ 与 $T_{p2}(N) = N^2$，当 $N < 10^6$ 时，二次算法仍可能更快。

3 一般分析规则 (General Rules)

1. **FOR loops**：循环时间至多为循环体和测试时间乘以迭代次数。
2. **Nested FOR loops**：嵌套循环中语句的总时间等于语句时间乘以各层循环规模的乘积。
3. **Consecutive statements**：连续语句时间相加，渐进上由最大项决定。
4. **IF/ELSE**：时间不超过条件测试时间加上两个分支中较大的时间。
5. **Recursions**：写出递推式，再求解递推式。

3.1 递归示例：Fibonacci

```
long int Fib( int N )
{
    if ( N <= 1 )
        return 1;
    else
        return Fib( N - 1 ) + Fib( N - 2 );
}
```

递推式为

$$T(N) = T(N-1) + T(N-2) + 2.$$

由于 $T(N)$ 至少按 Fibonacci 数增长，朴素递归 Fibonacci 的时间复杂度是指数级。课件强调：这个实现很差，根本原因是大量重复计算。

4 比较算法：最大子序列和 (Maximum Subsequence Sum)

4.1 问题定义

给定可能含负数的整数序列 $A_1, A_2, \ldots, A_N$，求

$$\max_{1 \le i \le j \le N} \sum_{k=i}^{j} A_k.$$

课件约定：如果所有整数都是负数，则最大和为 0。

4.2 算法 1：三重循环

枚举起点 i、终点 j，再循环求 A_i 到 A_j 的和：

```
MaxSum = 0;
for ( i = 0; i < N; i++ )
    for ( j = i; j < N; j++ ) {
        ThisSum = 0;
        for ( k = i; k <= j; k++ )
            ThisSum += A[k];
        if ( ThisSum > MaxSum )
            MaxSum = ThisSum;
    }
return MaxSum;
```

时间复杂度：$T(N) = O(N^3)$。

4.3 算法 2：二重循环并维护当前和

固定起点 i 后，终点 j 向右移动时复用前一次的和：

```
MaxSum = 0;
for ( i = 0; i < N; i++ ) {
    ThisSum = 0;
    for ( j = i; j < N; j++ ) {
        ThisSum += A[j];
        if ( ThisSum > MaxSum )
            MaxSum = ThisSum;
    }
}
return MaxSum;
```

时间复杂度：$T(N) = O(N^2)$。

4.4 算法 3：分治

最大子序列和只有三种可能：

1. 完全在左半部分。
2. 完全在右半部分。
3. 跨越中点。

左右两部分递归求解，跨中点部分线性扫描求解，因此：

$$T(N) = 2T(N/2) + cN, \qquad T(1) = O(1).$$

展开得：

$$T(N) = O(N \log N).$$

课件说明该结论对 $N \neq 2^k$ 时也成立。

4.5 算法 4：在线算法 (On-line Algorithm)

```
ThisSum = MaxSum = 0;
for ( j = 0; j < N; j++ ) {
    ThisSum += A[j];
    if ( ThisSum > MaxSum )
        MaxSum = ThisSum;
    else if ( ThisSum < 0 )
        ThisSum = 0;
}
return MaxSum;
```

时间复杂度：$T(N) = O(N)$。数组只扫描一次。

在线算法的含义

在任意时刻，算法都能对已经读入的数据给出当前正确答案。这就是课件中强调的 on-line algorithm。

4.6 四种算法对比

算法	思想	复杂度
Algorithm 1	枚举起点、终点并重新求和	$O(N^3)$
Algorithm 2	枚举起点、终点并递增维护和	$O(N^2)$
Algorithm 3	分治，处理左右与跨中点三类答案	$O(N \log N)$
Algorithm 4	扫描一次，负前缀和直接丢弃	$O(N)$

课件的运行时间表显示：当 N 从 10 增长到 100000 时，$O(N^3)$ 和 $O(N^2)$ 很快不可用，而 $O(N)$ 算法仍能保持很小运行时间。表中说明：读入输入的时间未计入。

5 运行时间中的对数 (Logarithms in the Running Time)

5.1 二分查找

给定有序数组：

$$A[0] \le A[1] \le \cdots \le A[N-1],$$

任务是查找 X。若 $X = A[i]$，返回 i；否则返回 -1。

```
int BinarySearch( const ElementType A[], ElementType X, int N )
{
    int Low, Mid, High;
    Low = 0; High = N - 1;
    while ( Low <= High ) {
        Mid = ( Low + High ) / 2;
        if ( A[Mid] < X )
            Low = Mid + 1;
        else if ( A[Mid] > X )
            High = Mid - 1;
        else
            return Mid;
    }
    return NotFound;
}
```

每次比较都把搜索范围减半，所以最坏时间复杂度为：

$$T_{\text{worst}}(N) = O(\log N).$$

课件提醒：二分查找适用于**静态且有序**的数据，例如字典查词。

5.2 欧几里得算法 (Euclid's Algorithm)

欧几里得算法用于求两个非负整数 m, n 的最大公约数 $\gcd(m, n)$。核心事实是：

$$\gcd(m, n) = \gcd(n, m \bmod n), \qquad n > 0.$$

因此每一步都可以把较大的问题转化成更小的问题，直到余数为 0。

```
unsigned int Gcd( unsigned int M, unsigned int N )
{
    unsigned int Rem;

    while ( N > 0 ) {
        Rem = M % N;
        M = N;
        N = Rem;
    }
    return M;
}
```

正确性与复杂度

- **不变式**：循环每轮开始时，当前两个数的最大公约数等于原问题的最大公约数。
- **终止条件**：当 $N = 0$ 时，$\gcd(M, 0) = M$，所以返回 M。
- **时间复杂度**：最坏情况下为 $O(\log \min(M, N))$。连续 Fibonacci 数会产生接近最坏的情况。
- **空间复杂度**：迭代版本只使用常数个变量，为 $O(1)$。

也可以写成递归形式：

```
unsigned int Gcd( unsigned int M, unsigned int N )
{
    if ( N == 0 )
        return M;
    return Gcd( N, M % N );
}
```

递归版本逻辑更短，但会使用递归栈，空间复杂度为 $O(\log \min(M, N))$。

5.3 快速幂 (Exponentiation by Squaring)

快速幂用于计算 X^N。如果直接连乘，需要 $N-1$ 次乘法，时间复杂度为 $O(N)$。快速幂利用：

$$X^N = \begin{cases} 1, & N = 0, \\ (X^{N/2})^2, & N \text{ 为偶数}, \\ (X^{(N-1)/2})^2 X, & N \text{ 为奇数}. \end{cases}$$

每一步都把指数大约减半，所以运行时间中会出现对数。

```
long int Pow( long int X, unsigned int N )
{
    if ( N == 0 )
        return 1;
    if ( N == 1 )
        return X;
    if ( N % 2 == 0 )
        return Pow( X * X, N / 2 );
    else
        return Pow( X * X, N / 2 ) * X;
}
```

快速幂的分析

- 每次递归把 N 变成 $\lfloor N/2 \rfloor$，递推式为

$$T(N) = T(N/2) + O(1).$$

- 解得时间复杂度为 $O(\log N)$，乘法次数远少于直接连乘。
- 上面的递归写法空间复杂度为 $O(\log N)$；若改成迭代写法，空间可降为 $O(1)$。
- 实际编程时要注意整数溢出；在很多题目中会改成取模快速幂，即每次乘法后对模数取余。

迭代快速幂常写作：

```
long int Pow( long int X, unsigned int N )
{
    long int Result = 1;

    while ( N > 0 ) {
        if ( N % 2 == 1 )
            Result *= X;
        X *= X;
```

```
        N /= 2;
    }
    return Result;
}
```

为什么它们都是对数级

二分查找、欧几里得算法和快速幂的共同点是：每一步都把问题规模显著缩小。二分查找把区间长度减半；快速幂把指数减半；欧几里得算法虽然不是简单减半，但余数序列下降很快，最坏情况也只需要对数级步数。

6 检查复杂度分析 (Checking Your Analysis)

6.1 倍增法

若 $T(N) = O(N)$，则通常有

$$\frac{T(2N)}{T(N)} \approx 2.$$

若 $T(N) = O(N^2)$，则通常有

$$\frac{T(2N)}{T(N)} \approx 4.$$

若 $T(N) = O(N^3)$，则通常有

$$\frac{T(2N)}{T(N)} \approx 8.$$

这个方法可用实验数据快速判断复杂度阶数。

6.2 归一化检查

若猜测 $T(N) = O(f(N))$，可观察

$$\frac{T(N)}{f(N)}$$

是否趋于稳定或有界。课件对应教材 Figures 2.12 和 2.13 的例子，核心思想是用数据验证增长阶数。

7 全章总结

7.1 复习主线

1. 先确认算法是否满足输入、输出、确定性、有穷性、有效性。
2. 分析时区分机器相关运行时间与机器无关复杂度。
3. 看到循环先数循环层数，看到递归先写递推式。
4. 掌握 O, Ω, Θ, o 的定义和常用规则。

5. 对同一问题比较多种算法时，抓住增长阶数：$N^3 \to N^2 \to N\log N \to N$ 的改进非常关键。

6. 用倍增法和归一化方法检查自己的复杂度判断。

“算法分析关心的不是某一次跑得多快，而是规模变大时会怎样。”